UNIVERSIDAD CATOLICA DEL MAULE

# NeatHogar

## Aplicación móvil colaborativa para la organización del hogar

Lucas Moya
Emilio Letelier
Benjamín Vidal

Ingeniería Civil Informática
Electivo III – Desarrollo de Aplicaciones Móviles

Profesor Román Gajardo
25 de septiembre de 2026

UNIVERSIDAD CATOLICA DEL MAULE

**Índice de Contenidos**

**Introducción ..... 3**

Descripción del problema ..... 3

Objetivo general y objetivos específicos ..... 4

Alcance funcional ..... 5

Usuarios y roles ..... 7

Arquitectura propuesta ..... 8

**Conclusiones ..... 24**

UNIVERSIDAD CATOLICA DEL MAULE

## Introducción

## Descripción del problema

### ¿Qué problema o necesidad resuelve la aplicación?

En hogares donde viven varias personas, la distribución de tareas domésticas, compras y responsabilidades suele organizarse de manera informal mediante conversaciones o mensajes. Esto puede provocar olvidos, duplicación de tareas y una distribución poco equitativa de los quehaceres.

La aplicación busca centralizar esta organización en un solo lugar, permitiendo registrar tareas, asignarlas a integrantes, marcar actividades como completadas y visualizar el aporte de cada persona. Además, incorporará un sistema de puntaje que considere el esfuerzo o dificultad de cada tarea y la disponibilidad de los integrantes.

### ¿A quién está dirigida? (público objetivo)

La aplicación estará dirigida a personas que comparten un mismo hogar y necesitan coordinar responsabilidades comunes, por ejemplo:

- Familias.
- Parejas.
- Estudiantes que comparten vivienda.
- Roomies o grupos de convivencia.

El objetivo es facilitar la organización entre personas con diferentes rutinas, horarios y niveles de disponibilidad.

### ¿Por qué una app móvil es la solución adecuada para este problema?

Una aplicación móvil resulta adecuada porque las tareas y necesidades del hogar ocurren durante distintos momentos del día y requieren acceso rápido a la información.

Desde el teléfono, los integrantes podrán revisar tareas pendientes, agregar productos a una lista de compras, marcar actividades como realizadas y consultar el estado general del hogar desde cualquier lugar.

Además, al trabajar con información sincronizada, los cambios realizados por una persona podrán ser visualizados por los demás integrantes del hogar

**Contexto: ¿existe algo similar? ¿qué la diferencia?**

Actualmente existen aplicaciones orientadas a la organización de tareas domésticas. Por ejemplo, **Nipto** permite compartir tareas, asignarles puntos y visualizar la participación de los integrantes mediante un ranking semanal. **Sweepy** también permite compartir tareas, utilizar puntajes según dificultad y generar planificación considerando cuánto tiempo desea dedicar una persona a la limpieza.

La propuesta se diferenciará al integrar en una misma aplicación **tareas domésticas, lista de compras, sistema de puntaje y disponibilidad de cada integrante**. El objetivo no será solamente comparar quién realiza más actividades, sino ayudar a representar una distribución más equilibrada considerando que no todos los miembros del hogar cuentan con el mismo tiempo o carga diaria.

Ese último punto es probablemente el **diferenciador más importante** de la idea: no medir únicamente cantidad de tareas, sino relacionar el aporte con el esfuerzo de las actividades y la disponibilidad de cada persona.

## Objetivo general y objetivos específicos

Nuestro objetivo general es desarrollar una aplicación móvil colaborativa que permita organizar, distribuir y monitorear las tareas, compras y responsabilidades de un hogar, considerando la participación y disponibilidad de sus integrantes para favorecer una distribución más equitativa de los quehaceres domésticos.

Como objetivos específicos contamos con los siguientes 6;

1. **Implementar un sistema de gestión de tareas domésticas** que permita crear, asignar, visualizar y marcar tareas como completadas.
2. **Desarrollar una lista de compras compartida** que pueda ser consultada y actualizada por los distintos integrantes del hogar.
3. **Incorporar un sistema de puntuación** que asigne valores a las tareas según su dificultad o esfuerzo requerido.
4. **Registrar la disponibilidad o carga cotidiana de cada integrante**, con el fin de apoyar una distribución más equilibrada de las responsabilidades.
5. **Mostrar estadísticas de participación** que permitan visualizar el aporte realizado por cada integrante del hogar.

6. **Permitir la gestión de distintos tipos de usuario**, diferenciando entre un jefe de hogar y los demás integrantes.

## Alcance funcional

El alcance funcional de la aplicación define las principales capacidades que tendrá el sistema durante su desarrollo. En este caso, la aplicación estará enfocada en facilitar la organización colaborativa del hogar, permitiendo gestionar integrantes, tareas domésticas, compras compartidas, disponibilidad y participación de cada miembro.

**Funcionalidades principales:**

a) **Crear un núcleo familiar**
   Un usuario podrá crear un nuevo hogar y quedar registrado como administrador.
b) **Unirse a un núcleo familiar**
   Los usuarios podrán incorporarse a un hogar existente mediante una invitación o código de vinculación.
c) **Gestión de integrantes**
   El administrador podrá visualizar los miembros del hogar, invitar nuevos integrantes y administrar sus roles básicos.
d) **Gestión de tareas domésticas**
   Los usuarios podrán crear tareas, asignarlas a uno o más integrantes, agregar una descripción y establecer su nivel de dificultad o puntaje.
e) **Completar tareas**
   Los integrantes podrán marcar sus tareas como realizadas, registrando su participación dentro del hogar.
f) **Visualización de tareas por integrante**
   Permitirá consultar las tareas organizadas según la persona responsable.
g) **Visualización de tareas mediante calendario**
   Permitirá visualizar las tareas programadas según su fecha.
h) **Lista de compras compartida**
   Los integrantes podrán agregar, consultar y marcar productos como comprados.
i) **Sistema de puntuación**
   Las tareas entregarán puntos dependiendo de su dificultad o esfuerzo, permitiendo registrar el aporte realizado por cada integrante.
j) **Registro de disponibilidad**
   Cada integrante podrá indicar su nivel de disponibilidad o carga cotidiana para ayudar a distribuir las responsabilidades de forma más equilibrada.

k) **Estadísticas de participación**
Permitirá visualizar puntos acumulados, cantidad de tareas completadas y nivel de participación de cada integrante.

l) **Dashboard del hogar**
Mostrará un resumen general del núcleo familiar, sus integrantes y accesos a las principales secciones de la aplicación.

**Funcionalidades opcionales:**

a) **Notificaciones de tareas pendientes.**
b) **Recordatorios.**
c) **Tareas recurrentes.**
d) **Historial de tareas.**
e) **Ranking semanal o mensual.**
f) **Sistema de logros o insignias.**
g) **Filtros por integrante, estado o categoría.**
h) **Personalización visual del hogar o perfil.**

**Limitaciones o exclusiones explicitas:**

a) **Pagos o transferencias entre integrantes.**
b) **Control de cuentas bancarias.**
c) **Gestión financiera avanzada.**
d) **Sistema de mensajería interno.**
e) **Integración con dispositivos inteligentes del hogar.**
f) **Asignación completamente automática de tareas.**
g) **Inteligencia artificial para decidir responsabilidades.**
h) **Gestión simultánea de múltiples hogares por un mismo usuario.**

## Usuarios y roles

La aplicación estará dirigida a personas que comparten un mismo hogar y necesitan coordinar tareas, compras y responsabilidades de manera colaborativa.

Cada usuario pertenecerá a un núcleo familiar y contará con un rol que determinará las acciones que puede realizar dentro de la aplicación.

### Administrador del hogar

El administrador será el usuario encargado de crear y gestionar el núcleo familiar.

Sus principales funciones serán:

- Crear el núcleo familiar.
- Invitar nuevos integrantes.
- Visualizar y gestionar los miembros del hogar.
- Asignar o modificar roles.
- Crear y asignar tareas.
- Consultar estadísticas generales de participación.
- Participar en tareas y listas de compras.
- Gestionar aspectos básicos de organización del hogar.

### Miembro del hogar

El miembro será cualquier usuario que forme parte del núcleo familiar y participe en las actividades compartidas.

Sus principales funciones serán:

- Visualizar las tareas disponibles y asignadas.
- Marcar tareas como completadas.
- Crear tareas, según los permisos definidos.
- Agregar y marcar productos en la lista de compras.
- Registrar su disponibilidad.
- Consultar sus puntos y estadísticas.
- Visualizar la participación de los demás integrantes.

## Disponibilidad de los integrantes

La disponibilidad no será considerada un rol de sistema, sino una característica del perfil de cada usuario.

Cada integrante podrá registrar su nivel de disponibilidad o carga cotidiana, por ejemplo, por motivos laborales, académicos o personales. Esta información será utilizada como apoyo para lograr una distribución más equilibrada de las responsabilidades del hogar.

Así queda bien claro que **Administrador/Miembro** son los roles reales, mientras que "trabajador", "estudiante", etc. son características del perfil y no roles del sistema.

## Arquitectura propuesta

La aplicación utilizará una arquitectura cliente-servidor. El frontend móvil será desarrollado con **React Native y Expo**, mientras que el backend será desarrollado utilizando **NestJS**.

La aplicación móvil será responsable de mostrar las interfaces y permitir que los usuarios interactúen con las distintas funcionalidades. Por su parte, NestJS se encargará de procesar las solicitudes, aplicar la lógica del sistema y comunicarse con la base de datos.

Esta arquitectura permitirá que varios integrantes de un mismo núcleo familiar puedan consultar y modificar información compartida, como tareas, compras, puntajes y disponibilidad.

## Tecnologías que utilizaremos

**React Native:** desarrollo de las interfaces y funcionalidades de la aplicación móvil.

**Expo:** facilitará el desarrollo, ejecución y prueba de la aplicación en dispositivos móviles.

**TypeScript:** será utilizado tanto en el frontend como en el backend para mantener un código tipado y organizado.

**NestJS:** será utilizado para desarrollar el backend y proporcionar los servicios necesarios para gestionar usuarios, hogares, tareas, compras y estadísticas.

**PostgreSQL:** Será el motor de base de datos que se usará para gestionar este apartado del proyecto.

**Git y GitHub:** permitirán llevar el control de versiones y facilitar el trabajo colaborativo del equipo.

### Backend y almacenamiento

El backend desarrollado con NestJS funcionará como intermediario entre la aplicación móvil y la base de datos. Cuando un usuario realice una acción, como crear una tarea, completar una actividad o agregar un producto a la lista de compras, la aplicación enviará una solicitud al backend. NestJS procesará la información y realizará las operaciones correspondientes sobre la base de datos.

**La información almacenada incluirá principalmente:**

- Usuarios.
- Núcleos familiares.
- Integrantes.
- Tareas.
- Lista de compras.
- Puntajes.
- Disponibilidad.
- Historial de tareas completadas.

### Arquitectura General

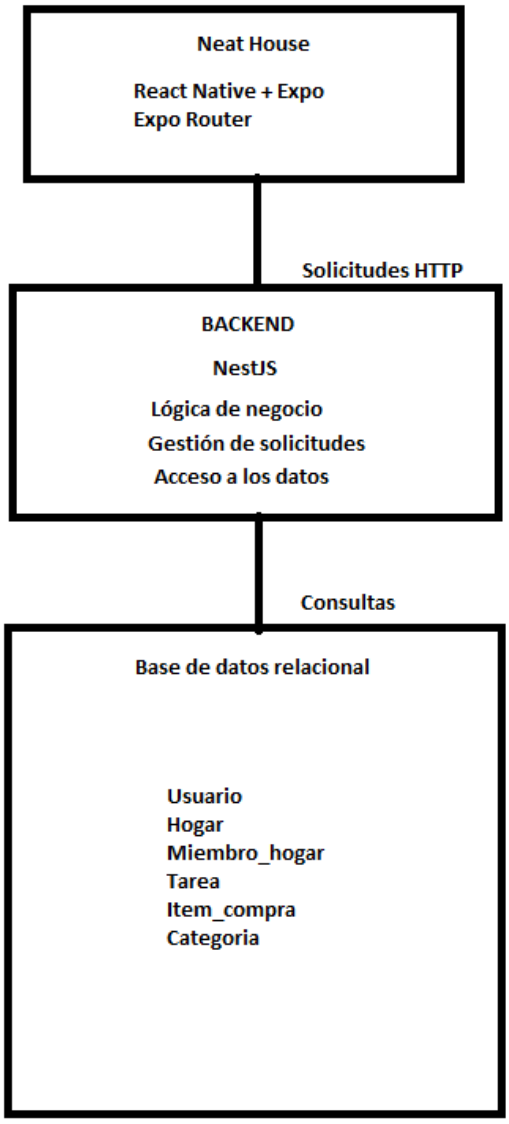
La arquitectura de la aplicación estará compuesta por tres capas principales: **aplicación móvil, backend y base de datos**.

La aplicación móvil será desarrollada con **React Native y Expo**, y será la encargada de mostrar las distintas pantallas y permitir la interacción de los usuarios.

El backend será desarrollado con **NestJS**, el cual recibirá las solicitudes provenientes de la aplicación móvil, procesará la lógica del sistema y gestionará el acceso a la información almacenada.

Finalmente, se utilizará una **base de datos relacional**, cuyo motor específico aún se encuentra por definir. Esta almacenará la información correspondiente a usuarios, hogares, integrantes, tareas, compras y categorías.

**El flujo general de comunicación será el siguiente:**




Cuando un usuario realice una acción dentro de la aplicación, como crear una tarea o agregar un producto a la lista de compras, React Native enviará una solicitud al backend desarrollado en NestJS. El backend procesará la solicitud y realizará las operaciones necesarias sobre la base de datos. Posteriormente, la información podrá ser consultada por los demás integrantes pertenecientes al mismo hogar.

## Justificación de las decisiones tecnológicas

La elección de **React Native** se debe a que permite desarrollar aplicaciones móviles utilizando una misma base de código para distintos sistemas operativos. Además, corresponde a la tecnología trabajada durante la asignatura, lo que facilita su implementación dentro del proyecto.

**Expo** será utilizado como entorno de desarrollo, ya que simplifica la ejecución y prueba de la aplicación en dispositivos móviles, permitiendo visualizar rápidamente los cambios realizados durante el desarrollo.

Para la navegación se utilizará **Expo Router**, debido a que permite organizar las pantallas mediante rutas y facilita el desplazamiento entre las distintas secciones de la aplicación, como Hogar, Tareas, Compras y Estadísticas.

El backend será desarrollado con **NestJS**, ya que permite estructurar de manera ordenada la lógica del servidor y gestionar las solicitudes realizadas desde la aplicación móvil. Además, utiliza TypeScript, lo que permite mantener una tecnología similar tanto en el frontend como en el backend.

Finalmente, se utilizará una **base de datos relacional**, debido a que la aplicación manejará información que se encuentra relacionada entre sí, como usuarios, hogares, integrantes, tareas, compras y categorías. El motor de base de datos específico será definido posteriormente por el equipo.

Esta elección tecnológica permite separar claramente la interfaz móvil, la lógica del sistema y el almacenamiento de datos, facilitando el desarrollo y mantenimiento de la aplicación.

# Diseño de Pantallas

Este apartado corresponde a un mockup hecho en **Figma** de la idea que tenemos como grupo de la aplicación sin antes haberla programado, los colores, componentes o número de vistas podrían cambiar eventualmente una vez empezado el desarrollo de la aplicación móvil.

Pantalla 1 - VinculacionPage



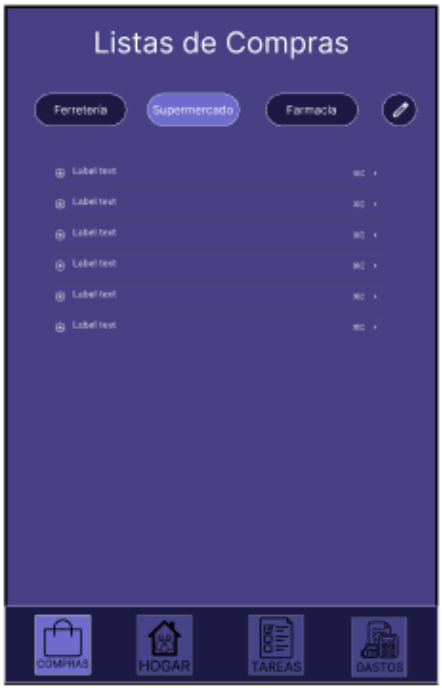

Pantalla 2 - DashboardPage




Pantalla 3 - AddMemberPage




Pantalla 4 - ComprasCompartidasPage



Pantalla 5.1 - TareasPage



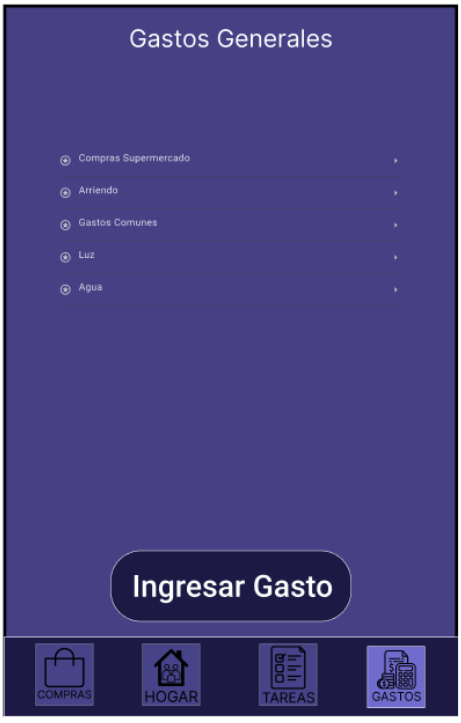

Pantalla 5.2 - TareasPage




Pantalla 6 - CreacionTareasPage




Pantalla 7 - GastosPage

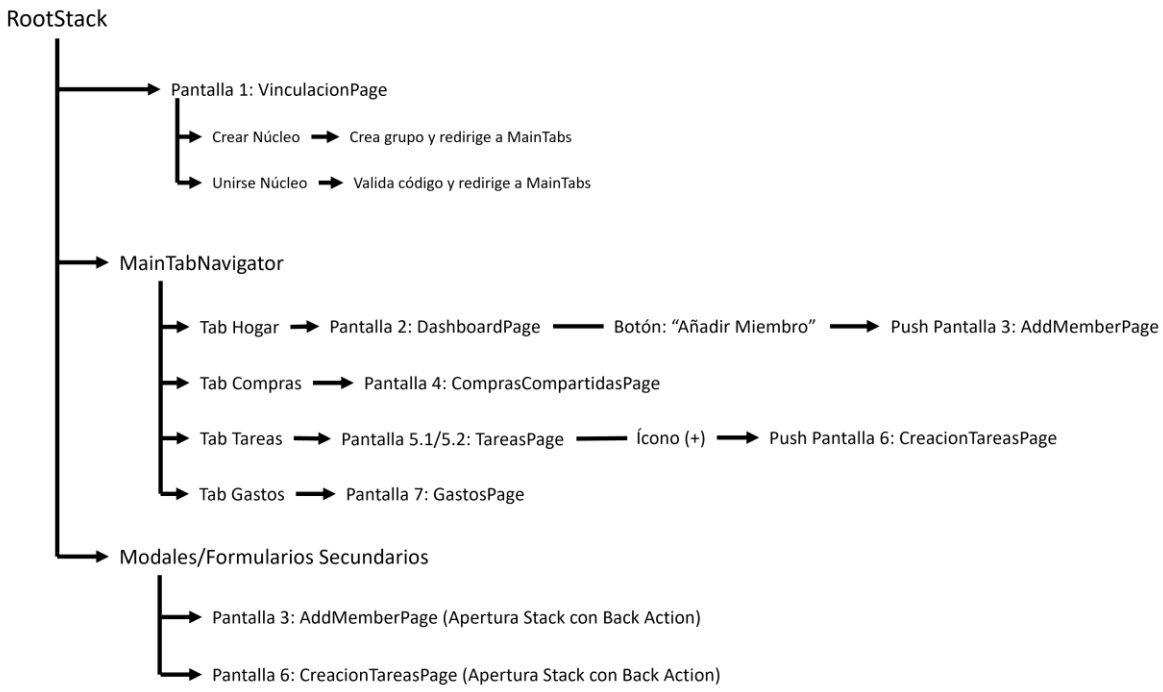


**Arquitectura General y Flujo de Navegación**

Para la navegación se implementará la librería estándar React Navigation (v6+), combinando dos patrones principales:

- Stack Navigator (RootStackNavigator): Esto gestionará la pila global de pantallas, el flujo inicial de vinculación/onboarding y la apertura de pantallas modulares/formularios secundarios (AddMemberPage, CreacionTareasPage).
- Bottom Tab Navigator (MainTabNavigator): Contenedor principal una vez el usuario está dentro de un hogar en la aplicación. Este manejará las 4 pestañas de acceso rápido persistentes en la barra inferior, esto se refiere a:
  1. Compras (ComprasCompartidasPage).
  2. Hogar (DashboardPage – Vista Principal).
  3. Tareas (TareasPage – Vistas 5.1 y 5.2).
  4. Gastos (GastosPage).

El flujo sería el siguiente:



**Desglose Técnico por Vista y Componentes React Native**

**Pantalla 1:** VinculacionPage (Onboarding/Acceso)

Objetivo: Permitir al usuario crear un nuevo usuario o unirse a uno existente mediante un código o enlace.

Flujo de navegación: Si no tiene un hogar activo, esta es la pantalla raiz. Al completar una acción con éxito se ejecuta *navitagion.replace('MainTabs').*

Componentess de React Native:

- *<SafeAreaView>* y *<View>*: Estructura y contenedor con fondo temático.
- *<Text>*: Títulos informativos y labels.
- *<Pressable> o <TouchableOpacity>: Las dos tarjetas cuadradas de acción principal (Crear Núcleo Familiar / Unirse a un Núcleo Familiar) y los botones inferiores (Importe sus Datos, Personaliza tu App).*
- *<Image>* o iconos vectoriales (*react-native-vector-icons / @expo/vector-icons*): Iconos de casa y enlace.

**Pantalla 2:** DashboardPage (Tab "Hogar" - Diario Mural)

Objetivo: Mostrar métricas visuales del hogar (resumen/progreso), listar integrantes activos diferenciando roles (*Administrador*, *Miembro*) y dar acceso para invitar nuevos miembros.

Flujo de navegación: Accesible desde la barra de tabs (Hogar). El botón *"Añadir Miembro"* dispara *navigation.navigate('AddMemberPage').*

Componentes clave de React Native:

- *<ScrollView>*: Permite desplazamiento vertical fluido si la lista de miembros crece.
- *<Svg> / <Path> (react-native-svg)*: Para renderizar la gráfica de métricas del diario mural.
- *<FlatList>*: Renderizado eficiente de las tarjetas de miembros del hogar.
- *<View>* estilizado: Tarjetas de miembros con avatar, nombre y etiqueta de rol.
- *<Pressable>*: Botón de acción "Añadir Miembro".

**Pantalla 3:** AddMemberPage (Añadir Miembro/Invitar)

Objetivo: Permitir al administrador incorporar nuevos integrantes al núcleo familiar mediante invitación.

Flujo de navegación: Se accede desde DashboardPage mediante el botón "Añadir Miembro". Al completar la invitación vuelve al panel principal del hogar.

Componentes React Native:

- *<SafeAreaView>:* Estructura segura de pantalla.
- *<TextInput>:* Ingreso de información del integrante.
- *<Picker>* o selector: Elección del tipo de miembro.
- *<Switch>:* Activación o configuración de opciones.
- *<Pressable>:* Botón "Enviar Invitación".
- *<Text>:* Etiquetas e instrucciones.

**Pantalla 4:** ComprasCompartidasPage (Tab "Compras")

Objetivo: Gestionar una lista de compras compartida entre todos los integrantes del hogar.

Flujo de navegación: Accesible desde la pestaña Compras. Permite agregar productos y actualizar su estado cuando sean comprados.

Componentes React Native:

- *<FlatList>:* Mostrar productos registrados.
- *<TextInput>:* Ingresar nuevos productos.
- *<Pressable>:* Agregar o modificar productos.
- *<Checkbox>* o *<Switch>:* Indicar si un producto fue comprado.

- *<View>:* Tarjetas de productos.
- *<Text>:* Nombre, cantidad y estado del producto.

**Pantalla 5.1 y 5.2:** TareasPage (Tab "Tareas" - Vista Segmentada)

Objetivo 5.1: Mostrar las tareas asignadas a cada integrante del hogar y visualizar la distribución de responsabilidades.

Flujo de navegación 5.1: Accesible desde la pestaña Tareas. El usuario puede seleccionar un integrante para consultar sus tareas asociadas.

Componentes React Native 5.1:

- *<FlatList>:* Listado de usuarios y tareas.
- *<View>:* Tarjetas de tareas.
- *<Text>:* Título, descripción y estado.
- *<Pressable>:* Abrir detalle o marcar como completada.
- *<ScrollView>:* Desplazamiento vertical.

Objetivo 5.2: Permitir visualizar las tareas organizadas según fechas programadas.

Flujo de navegación 5.2: Se accede desde la sección Tareas seleccionando la vista calendario.

Componentes React Native 5.2:

- *<View>:* Estructura del calendario.
- <Text>: Fechas y nombres de tareas.
- <Pressable>: Selección de días.
- Componentes externos de calendario si se requieren.
- *<ScrollView>:* Desplazamiento de eventos.

**Pantalla 6:** CreacionTareasPage (Formulario de Tarea)

Objetivo: Permitir crear nuevas tareas y asignarlas a un integrante del hogar.

Flujo de navegación: Se accede desde la lista de tareas mediante el botón "+".

Componentes React Native:

- *<TextInput>:* Título y descripción de la tarea.
- *<Picker>:* Selección del integrante asignado.
- *<Picker>:* Selección de categoría.
- *<Switch>:* Configuración del estado.
- *<Pressable>:* Botón confirmar.
- *<Text>:* Etiquetas del formulario.

**Pantalla 7:** GastosPage (Tab "Gastos")

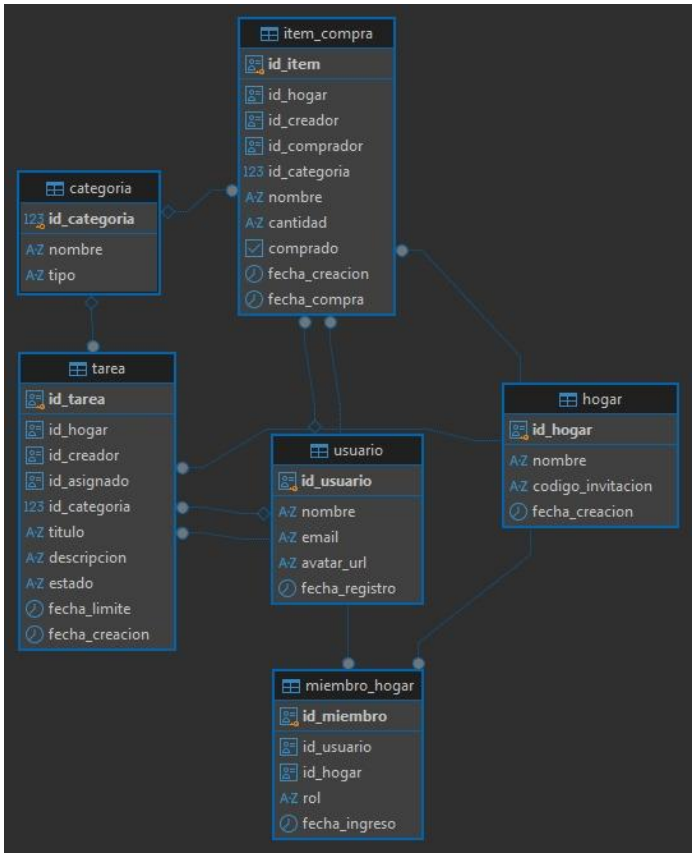
Objetivo: Mostrar la participación de los integrantes mediante puntos obtenidos, tareas completadas y distribución de responsabilidades.

Flujo de navegación: Accesible desde la barra inferior mediante la pestaña Estadísticas.

Componentes React Native:

- *<FlatList>:* listado de integrantes y sus resultados.
- *<View>:* tarjetas de estadísticas.
- *<Text>:* puntos, tareas realizadas y progreso.
- *<ProgressBar>* o *<View>* estilizado: representación visual del avance.
- *<Image>* o iconos: indicadores visuales.

Modelo de datos




El modelo de datos de **NeatHogar** fue diseñado para representar la relación entre los usuarios, los hogares y las principales funcionalidades de la aplicación, como la gestión de tareas y la lista de compras compartida.

La estructura propuesta corresponde a un **modelo relacional**, donde cada entidad almacena información específica y se relaciona con otras mediante identificadores.

Entidades principales:

**Usuario**

Representa a cada persona registrada en la aplicación.

**Atributos principales:**

- **id_usuario:** identificador único del usuario.
- **nombre:** nombre del usuario.
- **email:** correo electrónico.
- **avatar_url:** imagen de perfil.
- **fecha_registro:** fecha de creación de la cuenta.

**Hogar**

Representa un núcleo familiar o grupo de convivencia creado dentro de la aplicación.

**Atributos principales:**

- **id_hogar:** identificador único del hogar.
- **nombre:** nombre asignado al hogar.
- **codigo_invitacion:** código utilizado para incorporar nuevos integrantes.
- **fecha_creacion:** fecha de creación del hogar.

**Miembro_Hogar**

Relaciona a un usuario con un hogar y permite identificar el rol que cumple dentro de este.

**Atributos principales:**

- **id_miembro:** identificador del registro.
- **id_usuario**: usuario asociado.
- **id_hogar:** hogar al que pertenece.
- **rol:** rol que cumple el usuario, por ejemplo: administrador o miembro.
- **fecha_ingreso:** fecha en que se incorporó al hogar.

Esta entidad permite que la relación entre usuarios y hogares se gestione de manera independiente.

### Tarea

Almacena las tareas domésticas creadas dentro de un hogar.

**Atributos principales:**

- **id_tarea:** identificador único de la tarea.
- **id_hogar:** hogar al que pertenece.
- **id_creador:** usuario que creó la tarea.
- **id_asignado:** usuario responsable de realizarla.
- **id_categoria:** categoría asociada.
- **titulo:** nombre de la tarea.
- **descripcion:** información adicional.
- **estado:** indica si está pendiente o completada.
- **fecha_limite:** fecha máxima para realizarla.
- **fecha_creacion:** fecha de creación.

### Item_Compra

Representa cada producto registrado en la lista de compras compartida.

**Atributos principales:**

- **id_item:** identificador del producto.
- **id_hogar**: hogar al que pertenece.
- **id_creador:** usuario que agregó el producto.
- **id_comprador:** usuario que realizó la compra.
- **id_categoria:** categoría del producto.
- **nombre:** nombre del producto.
- **cantidad:** cantidad requerida.
- **comprado:** indica si el producto ya fue comprado.
- **fecha_creacion:** fecha en que fue agregado.
- **fecha_compra:** fecha en que fue marcado como comprado.

### Categoría

Permite clasificar las tareas y los productos de compra.

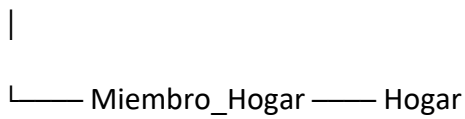
**Atributos principales:**

- **id_categoria:** identificador único.
- **nombre:** nombre de la categoría.
- **tipo:** permite diferenciar el uso de la categoría.

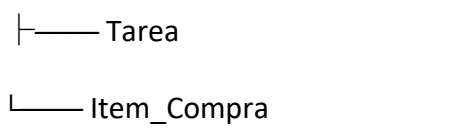
**Relaciones entre entidades**

Las principales relaciones del modelo son:

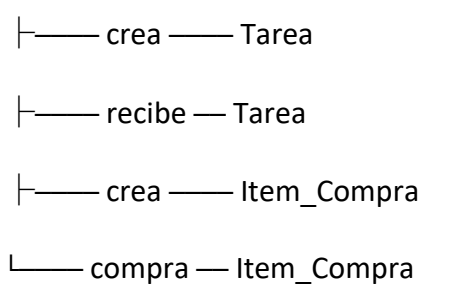
**Usuario**



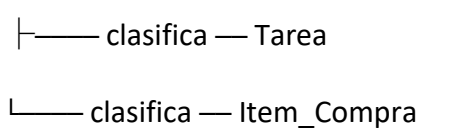
**Hogar**



**Usuario**



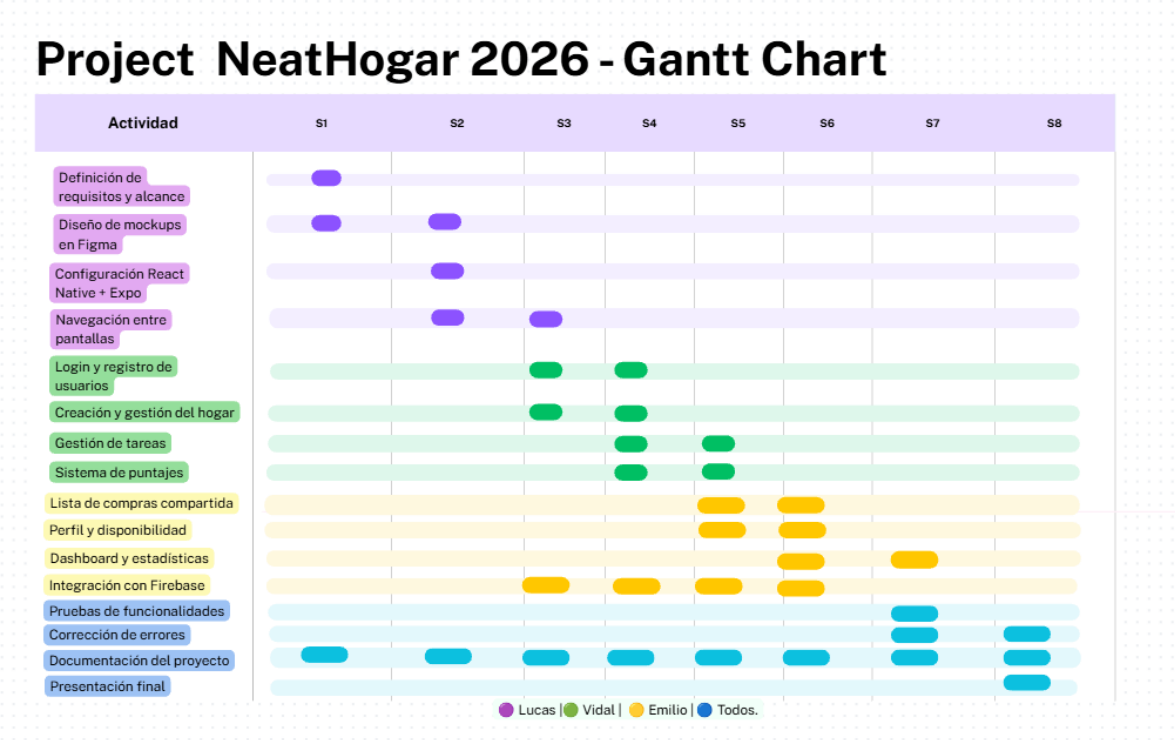
**Categoría**



Un **hogar puede tener varios integrantes**, mientras que cada registro de **Miembro_Hogar** vincula un usuario con un hogar y almacena su rol.

Un hogar también puede contener múltiples tareas y productos de compra. Las tareas permiten identificar tanto al usuario que las creó como al usuario al que fueron asignadas.

Planificación y distribución de trabajo




**Herramientas de colaboración:** se utilizará GitHub para el control de versiones del código, Trello para la organización y seguimiento de las actividades, Figma para el diseño de las interfaces y Visual Studio Code para el desarrollo de la aplicación.

# Conclusiones

El desarrollo de esta propuesta permitió definir de manera clara la idea de NeatHogar, una aplicación móvil orientada a mejorar la organización de tareas, compras y responsabilidades dentro de un hogar compartido. A partir del problema identificado, se establecieron los principales usuarios, funcionalidades y roles necesarios para que la aplicación pueda facilitar una distribución más ordenada y equilibrada de las actividades.

Además, se definieron elementos fundamentales para continuar con el desarrollo del proyecto, como la arquitectura de la aplicación, el diseño de pantallas, el modelo de datos y la planificación del trabajo. Con esto, NeatHogar queda estructurado como un proyecto realizable durante el semestre y con una base suficiente para comenzar su implementación en React Native con Expo, cumpliendo con los principales requerimientos planteados para la evaluación.

# Referencias

Atlassian. (2026). *Getting started with Trello*. Atlassian Support.

Expo. (2026). *Expo Router*. Expo Documentation.

Figma. (2026). *Figma Learn: Centro de ayuda*. Figma.

Firebase. (2026). *Cloud Firestore*. Google.

Firebase. (2026). *Firebase Authentication*. Google.

GitHub. (2026). *GitHub Docs*. GitHub.

Meta Platforms, Inc. (2026). *React Native documentation*. React Native.

Nipto. (2026). *Nipto: Split household chores*.

Sweepy. (2026). *Sweepy: Home cleaning schedule*.